

Characterizing Network Traffic with Programmable Dataplanes

Giuseppe Capasso

May 4, 2026

LLM Usage

LLMs have been used to generate part of the code of this assignment and as a general Python documentation search engine. In particular:

- Distribution fitting: LLM helped when fitting the bimodal creating the code to adapt the distribution left peak and right peak;
- *controller.py*: LLM generated the function to retrieve the register using *thrift* interface from the CLI.
- P4: LLM helped me in defining the flow tracking using the "hash" function and in adapting the code from p4 tutorial.
- Plotting: LLM generated part of the code to visualize logarithmic durations and fix the "zero-duration" bug, avoid $\log(0)$

LLMs have not been used to write this document.

1 Introduction

In this assignment, I analyzed a 15 minutes long network capture with two different methodologies:

- **Software based characterization:** I analyzed the data coming from the network *simply* converting the original PCAP in a CSV;
- **In-Network based analysis:** I extracted the feature directly on the data-plane using a *P4* program and *bmw2* switch;

This document contains a comparison between both methodologies, for further technical details refer to the other artifacts of the repository¹.

The rest of document is organized as follows. §2 discusses the general results from the analysis. §3 presents differences between the two methodologies.

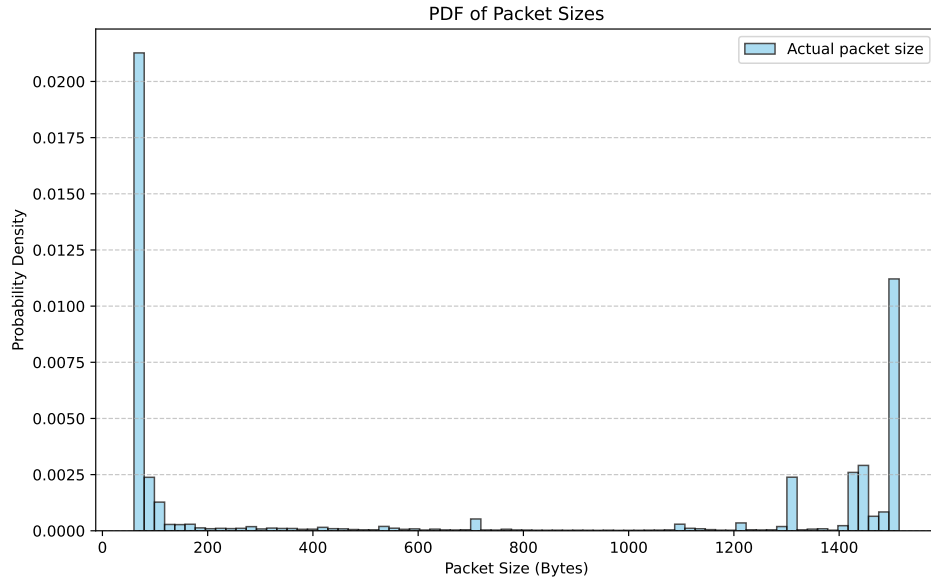
2 Analysis and Results

In this section, I explain results obtained using the full dataset.

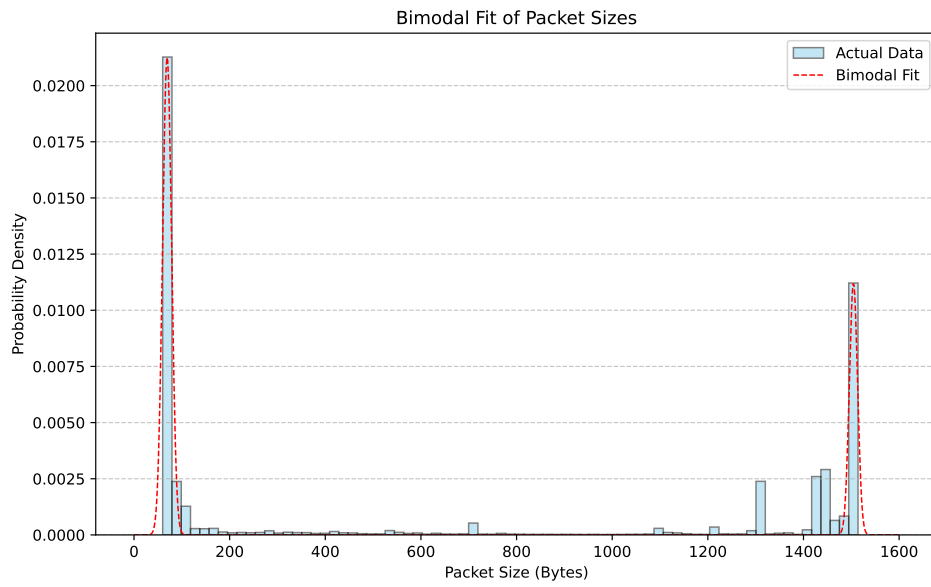
2.1 Packet-level analysis

As a packet level feature, I analyzed the packet sizes. The PDF is shown in Figure 1a. As in classical network environments, packet size is limited by the Maximum Transmission Unit (MTU) which is 1500. The behaviour is that the most majority of the traffic has either short lived messages (ICMP and other control/signaling protocols) or near to the MTU.

¹Repository is publicly available on Github at <https://github.com/alarmfox/imdea-assignment>



(a) Packet-size distribution



(b) Packet-size distribution

Figure 1: Analysis of network packet sizes: raw data vs. statistical fit.

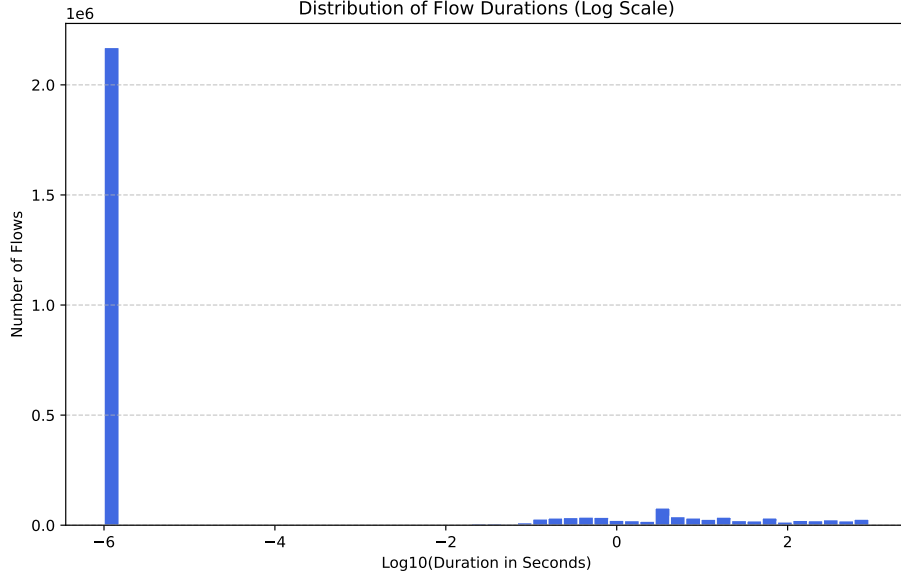


Figure 2: Log-scale distribution of flow durations.

Distribution fitting Looking at the distribution in Figure 1a, I can clearly fit a **bimodal** distribution². I estimated parameter using `scipy curve_fit`. The result of this fitting is shown in Figure 1b. The goodness of fit is evaluated using R^2 (Equation 1) and $RMSE$ (Equation 4).

$0 \leq R^2 \leq 1$. A high R^2 tells we have a good fit. On the other hand, a low $RMSE$ tells we have a good fit. $RMSE$ is an easy to calculate factor, but it is sensitive when dealing outliers.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} \quad (1)$$

$\bar{y}$ is the mean of the initial dataset. y_i is a point of the initial dataset. f_i is a point of the fitted distribution.

SS_{res} is the sum of square of residual. It indicates how much data fitted in the model differs from the mean value of original data (we take always the sum of squares to remove negative values).

$$SS_{res} = \sum_{i=1}^n (y_i - f_i)^2 \quad (2)$$

SS_{tot} is the total sum of squares. It is obtained with the difference between y_i and $\bar{y}$

$$SST = \sum_{i=1}^n (y_i - \bar{y})^2 \quad (3)$$

$RMSE$ is the mean distance between predicted data and original data squared:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - f_i)^2} \quad (4)$$

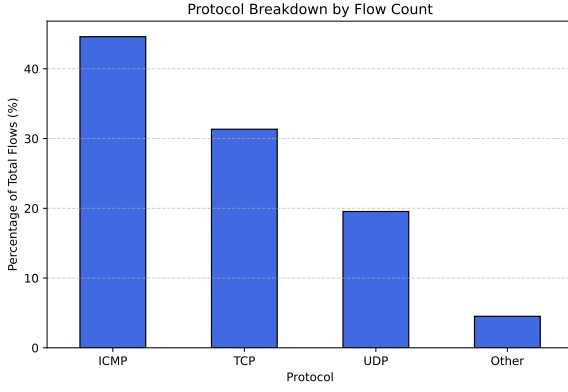
In this case, $R^2 > 0.9$ so the ‘curve_fit’ parameters are a good approximation of the initial dataset.

2.2 Flow-level analysis

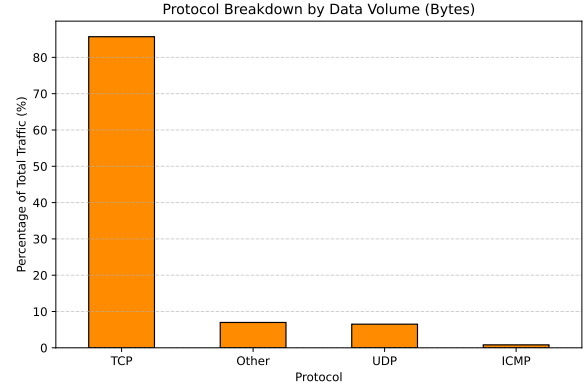
As flow level metrics, I analyzed the flow duration and analyzed TCP and UDP specific characteristics. Looking at flow duration distribution, Figure 2 shows that the vast majority of flows (both TCP and UDP) are short lived (many of them have 0 second duration).

To further understand the nature of this phenomenon, I classify the flows by protocol (Figure 3a) and by data exchanged (Figure 3b). Results tells that:

²In this simple case, the bi-modal distribution has been generated as the sum of two normal distributions.

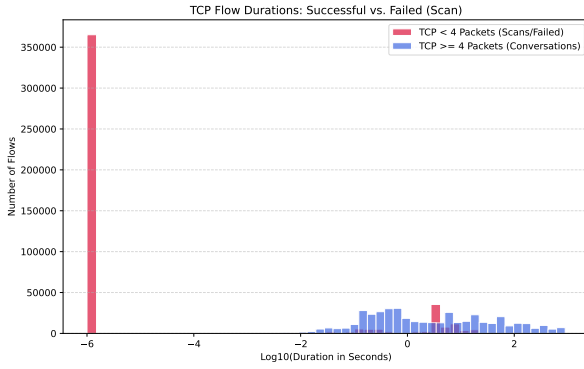


(a) Flow classification by protocol

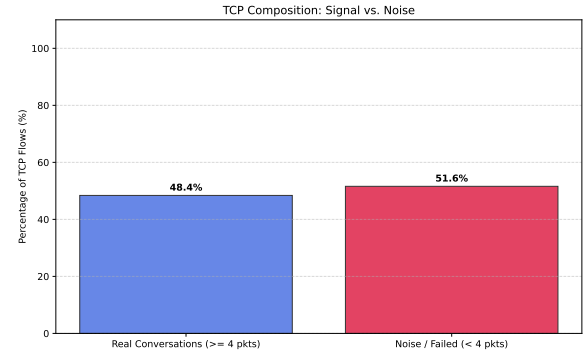


(b) Flow classification by bytes exchanged

Figure 3: Flow classification by protocol and bytes sent



(a) TCP durations: $packet \geq 4$ vs $packet < 4$



(b) Percentage of TCP flows with $packet \geq 4$ vs $packet < 4$

Figure 4: TCP flow analysis: broken handshake vs successful connections

- 44% of the traffic is ICMP and it exchanges less than 10% of data;
- 31% of the traffic is TCP and it exchanges more than 80% of data;
- 19% of the traffic is UDP and it exchanges 6% of data.

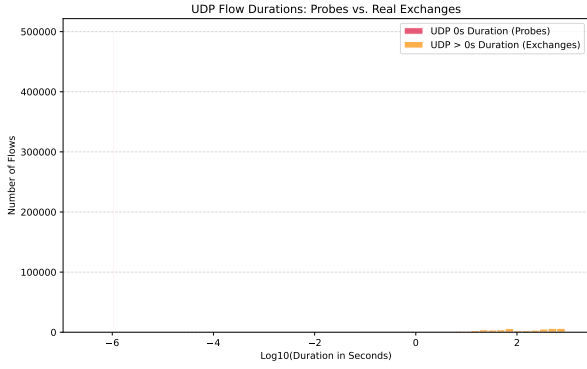
Removing the ICMP noise, it is clear that TCP traffic with low duration is *strange*. So I analyzed the TCP flows with less than 4 packets (as it indicates that the handshake was not completed). Figure 4a shows that the low durations TCP connections are the ones that do not complete handshake. Figure 4b shows that 51% of TCP connections are not valid. This is an indication that there might be a network scan going on.

I performed a similar analysis on UDP flows. In this case, a zero second UDP flows could mean either a single packet with no answer (eg. a failed DNS query) or a probe. Similarly to TCP, Figure 5a and Figure 5b shows that the 87% of UDP flows have zero duration.

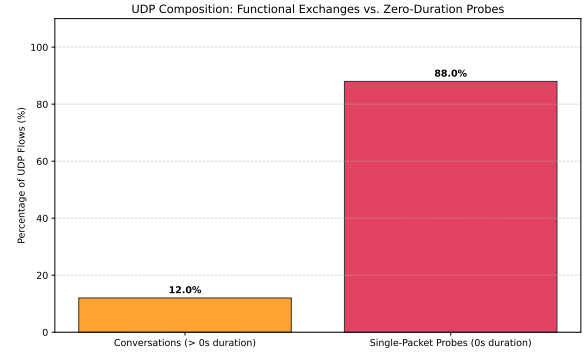
Removing the noise from ICMP traffic, invalid TCP and UDP connection, Figure 6 shows the cleaned-up network profile. Most of TCP and UDP connections last from 10 – 100

2.3 Security considerations

The high volume of ICMP traffic combined with a vast number of incomplete TCP handshakes strongly suggests active network reconnaissance (scanning). The prevalence of zero-second UDP flows further supports this, as they likely represent probes to closed ports. To confirm, Deep Packet Inspection (DPI) or a port-frequency analysis is recommended to identify the specific nature of the UDP and ICMP traffic (e.g., DNS vs. unusual probing).



(a) UDP flow durations: failed vs success



(b) Percentage of zero duration flows

Figure 5: UDP flow analysis: probes/failed vs successful connections

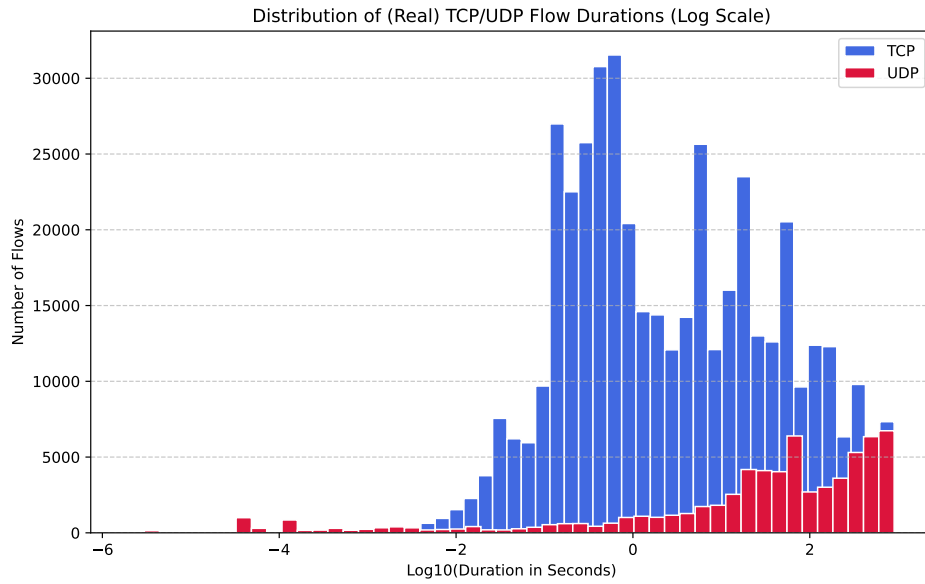
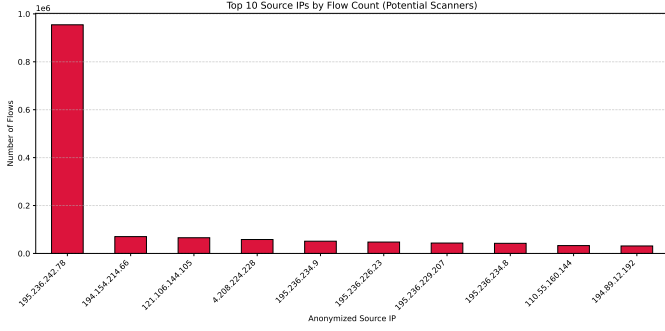
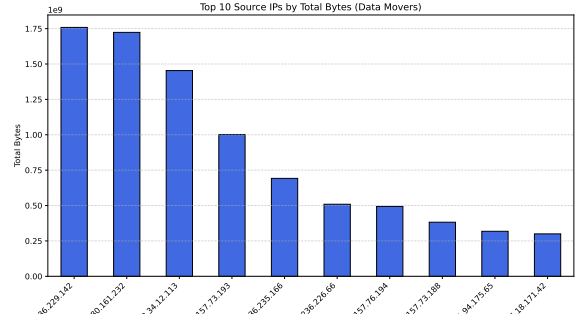


Figure 6: De-noised network profile



(a) Top 10 IP to initiate flows



(b) Top 10 IP by total bytes

Figure 7: Scanner IP detection.

To identify the source of the potential network scan, I analyzed the traffic profile of individual source IPs³. The underlying hypothesis is that a network scanner exhibits a highly asymmetric traffic pattern: it initiates a massive number of connections (flows) to probe ports, but transmits virtually no application data.

To test this, I aggregated the data by Source IP (ip.src) to calculate both the total flow count and the total bytes transmitted per host. By comparing the top 10 most active IPs by flow count against the top 10 IPs by data volume, we can isolate the scanner. Any IP that dominates the flow count but is entirely absent from the top data exchangers is confirmed to be exhibiting scanning behavior.

According to the plots shown below, the suspicious IP is **195.236.242.78**. It starts the majority of flows (Figure 7a), but it does not appear in the top 10 data movers (Figure 7b).

3 Methodologies comparison: Software-Based vs In-Network Classification

The analysis performed using the **bmv2** virtual switch extract features at line rate. Results are comparable to what has been shown §2⁴.

For example, Figure 8 shows how results are similar to both methodologies. Figure 9a and Figure 9b also show comparable results between different methodologies.

A comparison between the two methodologies is summarized in Table 1.

Throughput and Storage Efficiency Moving from software analysis to in-network classification completely changes how we handle performance and data. When relying on Python and PCAP files, the system hits a bottleneck quickly. Parsing times and I/O constraints simply don't scale well as traffic spikes. On top of that, capturing full payloads for deep packet inspection creates a massive storage burden, generating gigabytes of raw data in minutes. The P4 approach solves this by operating directly at line rate, letting the hardware push terabit speeds. Because it only looks at headers and extracts metadata summaries, it shrinks those gigabytes of traffic down into tiny, manageable CSV files.

Detection Latency and Actionability The Python approach is ultimately a post-mortem tool. It's great for forensics, but by the time we analyze the report, the attack is already over. The in-network method, on the other hand, processes packets live. Since the switch extracts features while the traffic is actively flowing, it's possible to build automated feedback loops. If a port scanner appears, the controller can instantly deploy a dynamic firewall rule to drop the packets.

Programmability and Architectural Constraints Software provides us with almost total freedom; we can readily utilize floating-point mathematics, comprehensive statistical libraries, and stateful objects to track network behavior. The P4 data plane offers none of these advantages. It is inherently rigid and

³I know that IP addresses are anonymized, but this analysis leverages the fact that common tools (like 'tcpdump' or 'editcap') use a consistent mapping.

⁴Due to performance limitation of the *bmv2* switch, *tcpdump* shows a 5% packet loss.

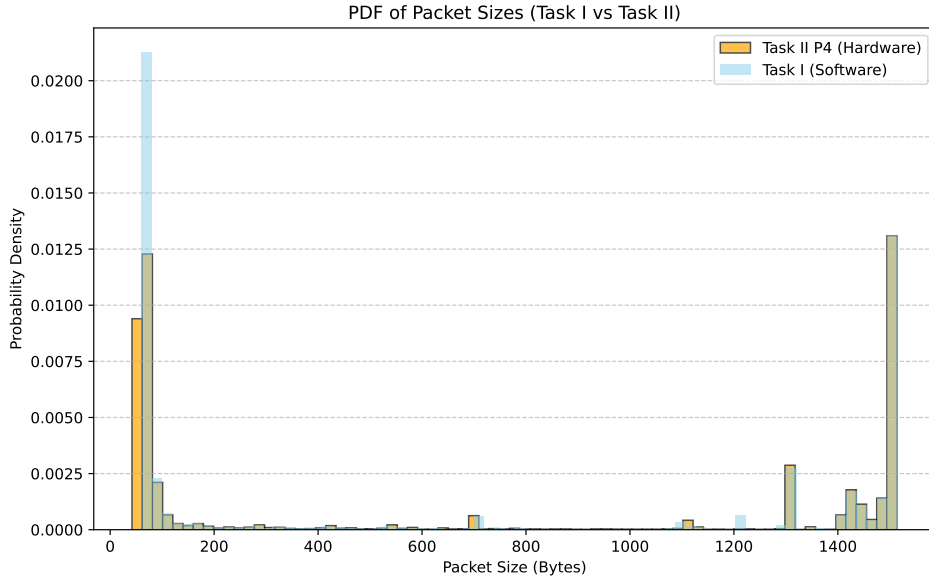
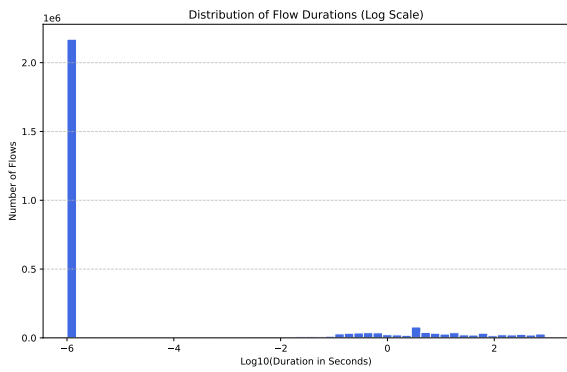
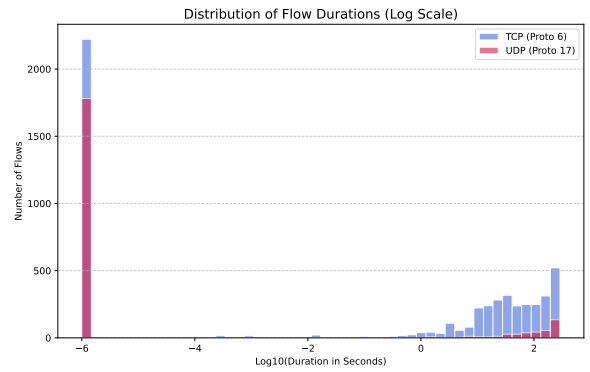


Figure 8: Packet size distribution comparison between Task 1 and Task 2



(a) *Software-based* methodology



(b) *In-network* methodology

Figure 9: Comparison between flow duration using both methodologies

Table 1: Comparison of Software-Based (Post-Mortem) and In-Network (P4) Classification

Feature	Software-Based (Python/PCAP)	In-Network (P4/BMv2)
Timing	Post-mortem (analysis after attack).	Real-time (during packet processing).
Storage	High: Gigabytes of raw PCAP files.	Minimal: Tiny CSVs of aggregated stats.
Throughput	Bottlenecked by I/O and parsing time.	Line rate (capable of Terabit speeds).
Flexibility	High: Complex libraries, floating point.	Low: Limited by ALU and hardware stages.
Statefulness	Inherently stateful (Python objects).	Stateless: Requires registers for state.
Data Granularity	Deep Packet Inspection (full payload).	Header-only / Metadata summaries.
Actionability	Forensic reporting only.	Feedback loop (Dynamic firewall rules).
Scalability	Poor: CPU intensive as traffic grows.	Excellent: Performance is hardware-bound.

fundamentally stateless. To maintain line-rate speeds, programmable switches are constrained by limited ALUs and strict pipeline stages, meaning that complex mathematical operations must be significantly simplified. Even tracking a standard multi-packet connection requires us to manually construct state using hardware registers.